

Algorytmy i Struktury Danych – Kompendium

1. Złożoność obliczeniowa i analiza algorytmów

Pytanie 1.1

Jakie jest rozwiązanie równania $T(n) = 2T(n/2) + n$ według Master Theorem?

Master Theorem stosujemy do rekurencji postaci $T(n) = aT(n/b) + f(n)$.

W naszym przypadku: $a = 2, b = 2, f(n) = n$.

Obliczamy próg: $n^{\log_b a} = n^{\log_2 2} = n^1 = n$

Ponieważ $f(n) = n = \Theta(n^{\log_b a})$, jesteśmy w **Przypadku 2** Master Theorem:

$$\boxed{T(n) = \Theta(n \log n)}$$

Jest to złożoność m.in. algorytmu Merge Sort.

Pytanie 1.2

Co dokładnie bada analiza zamortyzowana?

Analiza zamortyzowana bada **średni koszt operacji w sekwencji n operacji**, a nie koszt pojedynczej operacji w izolacji.

Kluczowe cechy:

- Uwzględnia, że **droga operacja zdarza się rzadko** i jest „rozkładana” na tanie operacje
- Wynik to **górne ograniczenie na całkowity koszt** sekwencji – bardziej precyzyjne niż mnożenie najgorszego przypadku przez n
- Popularne metody: **metoda kosztu zagregowanego**, **metoda potencjału (Bankowska)**, **metoda kredytowa**

Przykład: tablica dynamiczna – pojedyncze wstawianie zajmuje $O(n)$ gdy trzeba realokować, ale zamortyzowany koszt jednego wstawiania wynosi $O(1)$.

Pytanie 1.3

Jak poprawnie uporządkować rosnąco złożoności: $n \log n, n^2, n^3, 2^n$?

$$n \log n < n^2 < n^3 < 2^n$$

Złożoność	Klasa	Charakter
$n \log n$	Polilogarytmiczna	Algorytmy „dziel i zwyciężaj”
n^2	Kwadratowa	Algorytmy naiwne, sortowania proste
n^3	Sześcienna	Mnożenie macierzy, Floyd-Warshall
2^n	Wykładnicza	Przeszukiwanie brute-force

Dla $n = 1000$: różnica między $n \log n$ ($\approx 10\,000$) a 2^n (liczba astronomiczna) jest kolosalna.

2. Algorytmy sortowania i wyszukiwania elementu

Pytanie 2.1

Jaka jest złożoność Insertion Sort w najgorszym przypadku?

$$\boxed{O(n^2)}$$

Najgorszy przypadek zachodzi, gdy tablica jest **posortowana malejąco** – każdy nowy element musi być przestawiony przez całą już posortowaną część. Liczba porównań wynosi $1 + 2 + 3 + \dots + (n-1) = n(n-1)/2 = O(n^2)$.

W **najlepszym przypadku** (tablica już posortowana) złożoność wynosi $O(n)$ – dlatego Insertion Sort bywa używany do małych lub prawie posortowanych danych.

Pytanie 2.2

Czego używa zrandomizowany Quicksort do wyboru pivot'a?

Zrandomizowany Quicksort wybiera element osiowy (pivot) **losowo** – za pomocą generatora liczb pseudolosowych spośród aktualnie przetwarzanego podzakresu tablicy.

Cel: eliminacja najgorszego przypadku $O(n^2)$, który w deterministycznym Quicksorcie (np. pivot = ostatni element) zachodzi dla danych już posortowanych. Dzięki losowemu wyborowi **oczekiwana** złożoność wynosi **$O(n \log n)$** niezależnie od rozkładu danych wejściowych.

Pytanie 2.3

Czy Merge Sort jest algorytmem deterministycznym?

☑ **TAK** – Merge Sort jest w pełni deterministyczny

Zawsze wykonuje dokładnie te same operacje na tych samych danych wejściowych, niezależnie od kolejnych uruchomień. Złożoność wynosi **$\Theta(n \log n)$** w każdym przypadku (najlepszym, średnim i najgorszym).

Pytanie 2.4

Jaka jest złożoność wyszukiwania binarnego?

☑ **$O(\log n)$**

Wyszukiwanie binarne działa na **posortowanej tablicy** i w każdym kroku redukuje przestrzeń poszukiwań o połowę. Po k krokach pozostaje $n/2^k$ elementów – kończymy gdy $2^k \geq n$, czyli $k = \lceil \log_2 n \rceil$.

Pytanie 2.5

Który algorytm sortowania jest przykładem metody „dziel i zwyciężaj”?

Metoda **dziel i zwyciężaj** polega na: podziale problemu, rekurencyjnym rozwiązaniu podproblemów i scaleniu wyników.

Klasyczne przykłady algorytmów sortowania:

Algorytm	Dziel i zwyciężaj?	Złożoność
Merge Sort	☑ TAK	$\Theta(n \log n)$
Quicksort	☑ TAK	$O(n \log n)$ śr.
Insertion Sort	☑ NIE	$O(n^2)$
Selection Sort	☑ NIE	$O(n^2)$

Pytanie 2.6

Za pomocą jakiej struktury/algorytmu można znaleźć k-ty najmniejszy element w oczekiwanym czasie liniowym?

☑ **Algorytm Quickselect (selekcja oparta na partycjonowaniu Quicksort)**

Działa analogicznie do Quicksort, ale po wybraniu losowego pivot'a i partycjonowaniu **rekuruje tylko w jedną stronę** – tę, w której znajduje się k-ty element.

- **Oczekiwana złożoność:** $O(n)$
- **Najgorszy przypadek:** $O(n^2)$ – redukowany przez losowy wybór pivot'a
- Alternatywa deterministyczna: algorytm **Median of Medians** gwarantuje $O(n)$ w najgorszym przypadku

Pytanie 2.7

Czy Insertion Sort jest algorytmem stabilnym?

☑ TAK – Insertion Sort jest stabilny

Elementy o równych kluczach zachowują swój oryginalny względny porządek, ponieważ algorytm wstawia nowy element **za** już istniejącymi równymi elementami (warunek $a[j] > key$, a nie $\geq$).

3. Struktury danych: Drzewa (BST, AVL, Czerwono-Czarne)

Pytanie 3.1

W jakiej kolejności przegląd inorder w BST odwiedza węzły?

Przebieg **inorder** (LNP – lewe poddrzewo → węzeł → prawe poddrzewo):

☑ W BST inorder odwiedza węzły w kolejności niemalejącej (rosnącej)

```
    5
   / \
  3   7
 / \
2  4
```

Inorder: $2 \rightarrow 3 \rightarrow 4 \rightarrow 5 \rightarrow 7$

Właśność ta wynika wprost z definicji BST: lewy potomek < rodzic < prawy potomek.

Pytanie 3.2

Jaka jest w najgorszym przypadku wysokość drzewa BST?

☑ $O(n)$

Najgorszy przypadek zachodzi przy wstawianiu elementów w **posortowanej kolejności** – drzewo degeneruje do **liniowej listy** (jeden długi łańcuch). Wszystkie operacje (wyszukiwanie, wstawianie, usuwanie) degenerują do $O(n)$.

Dlatego stosuje się drzewa **zrównoważone** (AVL, czerwono-czarne), które gwarantują $O(\log n)$.

Pytanie 3.3

Jaki jest koszt wyszukiwania w zrównoważonym drzewie BST?

☑ $O(\log n)$

W drzewie zrównoważonym wysokość $h = O(\log n)$, a wyszukiwanie schodzi co najwyżej o jeden poziom na każdy krok. Gwarantują to m.in. drzewa AVL i czerwono-czarne.

Pytanie 3.4

Jaka jest maksymalna wysokość drzewa AVL z n węzłami?

☑ $O(\log n)$, dokładniej: $h \leq 1,44 \cdot \log_2(n + 2) - 1,328$

Drzewo AVL utrzymuje **warunek AVL**: dla każdego węzła wysokości lewego i prawego poddrzewa różnią się co najwyżej o 1. Gwarantuje to logarytmiczną wysokość – w odróżnieniu od niezrównoważonego BST.

Po każdej operacji wstawiania/usuwania wykonuje się co najwyżej $O(\log n)$ **rotacji** przywracających równowagę.

Pytanie 3.5

Jakiego koloru muszą być korzeń i liście (węzły NIL) w drzewie czerwono-czarnym?

❑ Zawsze CZARNE

Własności drzewa czerwono-czarnego:

1. Każdy węzeł jest czerwony lub czarny
2. **Korzeń jest czarny**
3. **Każdy liść (węzeł NIL/strażnik) jest czarny**
4. Czerwony węzeł może mieć tylko czarne dzieci (brak dwóch czerwonych z rzędu)
5. Każda ścieżka od węzła do liścia zawiera tę samą liczbę czarnych węzłów (**czarna wysokość**)

Własności te gwarantują, że wysokość drzewa wynosi co najwyżej $2 \cdot \log_2(n+1) = O(\log n)$.

4. Struktury danych: Kopce

Pytanie 4.1

Jaki jest koszt operacji decrease-key w kopcu Fibonacciego?

❑ $O(1)$ **zamortyzowany**

To kluczowa przewaga kopca Fibonacciego nad innymi kopcami. Operacja decrease-key w kopcu:

Kopiec	decrease-key
Binarny	$O(\log n)$
Dwumianowy	$O(\log n)$
Fibonacciego	$O(1)$ zamortyzowany

Dzięki temu algorytm Dijkstry z kopcem Fibonacciego osiąga złożoność $O(V \log V + E)$.

Pytanie 4.2

Z jaką złożonością buduje się kopiec z n elementów?

❑ $O(n)$ (metoda bottom-up / buildHeap)

Choć intuicyjnie n wywołań insert kosztowałoby $O(n \log n)$, procedura **buildHeap** (naprawianie właściwości kopca od dołu ku górze) działa w czasie $O(n)$. Wynika to z tego, że większość węzłów ma małą wysokość – węzłów na poziomie h jest $\leq \lceil n/2^{(h+1)} \rceil$, a każde przesiewanie kosztuje $O(h)$.

5. Struktury danych: Tablice mieszające (Hashing)

Pytanie 5.1

Czym jest kolizja w tablicy mieszającej?

Kolizja zachodzi, gdy dwa różne klucze $k_1 \neq k_2$ zostają zmapowane przez funkcję mieszającą na ten sam indeks:

❑ $h(k_1) = h(k_2)$

Metody rozwiązywania kolizji:

- **Łańcuchowanie (chaining):** każda komórka tablicy zawiera listę elementów
- **Adresowanie otwarte (open addressing):** szukamy kolejnej wolnej komórki (próbkowanie liniowe, kwadratowe, podwójne)

Pytanie 5.2

Czego przykładem jest próbkowanie kwadratowe (quadratic probing)?

❑ Jest to przykład adresowania otwartego (open addressing)

▮ Jest to przykład adresowania otwartego (open addressing)

W quadratic probing sekwencja próbkowania wygląda następująco:

$$h(k, i) = (h'(k) + c_1 \cdot i + c_2 \cdot i^2) \bmod m$$

gdzie $i = 0, 1, 2, \dots$ oznacza numer próby. Różni się od próbkowania liniowego tym, że kolejne pozycje nie są sąsiednie – redukuje to **clustering pierwotny**, choć może pojawić się **clustering wtórny**.

Pytanie 5.3

Jaki jest koszt wyszukiwania przy użyciu doskonałej funkcji mieszającej?

▮ $O(1)$ – czas stały

Doskonała funkcja mieszająca (perfect hash function) jest **iniekcją** – mapuje każdy klucz na unikalny indeks, więc kolizje w ogóle nie występują. Wyszukiwanie polega na jednym obliczeniu $h(k)$ i bezpośrednim dostępie do tablicy.

6. Struktury danych: Listy z przeskokami i tablice dynamiczne

Pytanie 6.1

Ile wynosi w najgorszym przypadku wysokość nowego węzła w skip list?

▮ $O(\log n)$ – z wysokim prawdopodobieństwem

Wysokość węzła jest wyznaczana losowo (rzuty monetą). Implementacje zazwyczaj ograniczają maksymalny poziom do $\lceil \log_2 n \rceil + 1$. Bez ograniczenia teoretyczny najgorszy przypadek to $O(n)$, ale prawdopodobieństwo przekroczenia $O(\log n)$ maleje wykładniczo.

Pytanie 6.2

Jaka jest złożoność wyszukiwania w skip list?

▮ $O(\log n)$ oczekiwany (z wysokim prawdopodobieństwem)

Skip list łączy zalety posortowanej listy z efektywnością przeszukiwania zbliżoną do drzewa binarnego. Wyszukiwanie postępuje od najwyższego poziomu ku dołowi, „przeskakując” duże fragmenty listy.

Pytanie 6.3

Jaki jest zamortyzowany czas wstawiania dla tablicy dynamicznej podwajającej swój rozmiar?

▮ $O(1)$ zamortyzowany

Analiza metodą kosztu zagregowanego: przy n wstawieniach łączny koszt wszystkich realokacji wynosi: $1 + 2 + 4 + 8 + \dots + n = 2n - 1 = O(n)$

Dzieląc przez n operacji: $O(n)/n = O(1)$ na operację.

Pytanie 6.4

Czym różnią się wskaźniki w skip list od wskaźników w standardowej liście jednokierunkowej?

Cecha	Lista jednokierunkowa	Skip list
Liczba wskaźników na węzeł	1 (tylko <code>next</code>)	wiele (jeden na każdy poziom)
Kierunek	Jeden następnik	Następnicy na różnych poziomach
Przeskakiwanie	Niemożliwe	Możliwe dzięki wyższym poziomom

W skip list węzeł na poziomie k ma k wskaźników – każdy „przeskakuje” inną liczbę elementów, co umożliwia efektywne wyszukiwanie.

7. Algorytmy grafowe: Najkrótsze ścieżki

Pytanie 7.1

Jaka jest złożoność Dijkstry z wyszukiwaniem liniowym (bez kolejki priorytetowej)?

| $\mathcal{O}(V^2)$

Bez kolejki priorytetowej przy każdej z V iteracji szukamy minimum liniowo wśród V wierzchołków $\rightarrow V \cdot \mathcal{O}(V) = \mathcal{O}(V^2)$. Efektywne dla **grafów gęstych** ($|E| \approx V^2$).

Pytanie 7.2

Jaki problem rozwiązuje algorytm Floyd-Warshalla?

| $\mathcal{O}(V^3)$ Problem najkrótszych ścieżek między wszystkimi parami wierzchołków (All-Pairs Shortest Paths, APSP)

Złożoność: $\mathcal{O}(V^3)$ – trzy zagnieżdżone pętle po V wierzchołkach.

Działa poprawnie także dla **ujemnych wag krawędzi** (ale nie dla ujemnych cykli). Używa programowania dynamicznego z pośrednikiem k : $\text{dist}[i][j] = \min(\text{dist}[i][j], \text{dist}[i][k] + \text{dist}[k][j])$.

Pytanie 7.3

W jakim grafie BFS znajduje najkrótsze ścieżki?

| $\mathcal{O}(V + E)$ W grafie nieważonym (lub o jednostkowych wagach krawędzi)

BFS eksploruje wierzchołki warstwami – najpierw odległość 1, potem 2, itd. Gwarantuje to znalezienie ścieżki o **minimalnej liczbie krawędzi**. Dla grafów ważonych należy użyć Dijkstry.

Pytanie 7.4

Jakich wag krawędzi i jakiego grafu wymaga algorytm Dijkstry?

| $\mathcal{O}(V \log V + E)$ Krawędzie o nieujemnych wagach; graf skierowany lub nieskierowany

Ujemne wagi łamią założenie Dijkstry (po "zamknięciu" wierzchołka nie możemy go poprawić). W takim przypadku należy użyć algorytmu **Bellmana-Forda** ($\mathcal{O}(VE)$).

Pytanie 7.5

Jaka jest złożoność Dijkstry z kopcem Fibonacciego?

| $\mathcal{O}(V \log V + E)$

Jest to asymptotycznie optymalny wariant Dijkstry. Osiąga lepszą złożoność niż kopiec binarny dzięki operacji decrease-key w czasie $\mathcal{O}(1)$ amortyzowanym.

Pytanie 7.6

Do jakiego algorytmu MST Dijkstra jest strukturalnie podobny?

| $\mathcal{O}(V \log V + E)$ Do algorytmu Prima

Oba algorytmy:

- Utrzymują zbiór „odwiedzonych” wierzchołków i kolejkę priorytetową
 - W każdym kroku wybierają wierzchołek o minimalnym kluczu
 - Różnią się tylko **definicją klucza**: Dijkstra $\rightarrow$ odległość od źródła; Prim $\rightarrow$ waga krawędzi łączącej z drzewem
-

Pytanie 7.7

Jaka jest złożoność Dijkstry z kopcem binarnym?

• $O((V + E) \log V)$

- V operacji `extract-min` : $O(V \log V)$
- E operacji `decrease-key` : $O(E \log V)$
- Łącznie: $O((V + E) \log V)$

Efektywne dla **grafów rzadkich** ($E \approx V$).

8. Algorytmy grafowe: Przeszukiwanie i MST

Pytanie 8.1

Do czego wykorzystuje się algorytm DFS?

DFS (Depth-First Search) ma szerokie zastosowania:

Zastosowanie	Opis
Sortowanie topologiczne	Kolejność wierzchołków w DAG
SCC	Silnie spójne składowe (algorytm Tarjana/Kosaraju)
Wykrywanie cykli	Śledzenie krawędzi powrotnych (back edges)
Mosty i punkty artykulacji	Krytyczne krawędzie/wierzchołki
Przeszukiwanie labiryntów	Eksploracja z powrotami

Złożoność: $\Theta(V + E)$

Pytanie 8.2

Za pomocą jakich algorytmów można znaleźć minimalne drzewo rozpinające?

• Algorytm Kruskala i Algorytm Prima

Cecha	Kruskal	Prim
Podejście	Krawędziowe	Wierzchołkowe
Struktura	Union-Find	Kolejka priorytetowa
Złożoność	$O(E \log E)$	$O(E \log V)$ z kopcem binarnym
Lepszy dla	Grafów rzadkich	Grafów gęstych

Pytanie 8.3

Jakich operacji na zbiorach używa algorytm Kruskala?

• Operacji Find i Union ze struktury Union-Find (Disjoint Set Union)

- **Find(x)**: zwraca reprezentanta zbioru zawierającego x – sprawdza, czy dwa wierzchołki są w tym samym składowym
- **Union(x, y)**: łączy dwa zbiory – wywołane po dodaniu krawędzi do MST

Z optymalizacją (kompresja ścieżek + łączenie według rangi): niemal $O(1)$ na operację $O(\alpha(n))$ gdzie α to odwrotna funkcja Ackermanna).

Pytanie 8.4

Jaka zależność między $|E|$ a $|V|$ zachodzi w drzewie?

▮ $|E| = |V| - 1$

Drzewo o n wierzchołkach ma dokładnie $n-1$ krawędzi. Jest to spójny graf acykliczny – usunięcie jednej krawędzi rozprowadza go, a dodanie tworzy cykl.

9. Algorytmy grafowe: Sieci przepływowe

Pytanie 9.1

Co dzieje się z przepustowością krawędzi po znalezieniu ścieżki powiększającej?

Po znalezieniu ścieżki powiększającej (augmenting path) o przepustowości residualnej f :

- Przepustowość krawędzi **w przód** zmniejsza się o f : $c(u, v) -= f$
- Przepustowość krawędzi **wstecznej** zwiększa się o f : $c(v, u) += f$

Aktualizuje się **sieć residualna (sieć reszt)**, co umożliwia algorytmowi cofnięcie wcześniejszych decyzji i znalezienie lepszego przepływu.

Pytanie 9.2

Do czego służy algorytm Forda-Fulkersona?

▮ Do wyznaczania maksymalnego przepływu w sieci przepływowej

Algorytm iteracyjnie znajduje ścieżki powiększające (od źródła s do ujścia t w sieci residualnej) i zwiększa przepływ, aż żadna ścieżka już nie istnieje.

Kluczowy wynik: **Twierdzenie o Max-Flow Min-Cut** – maksymalny przepływ = pojemność minimalnego przekroju.

10. Algorytmy tekstowe (dopasowywanie wzorców)

Pytanie 10.1

Który algorytm dopasowywania wzorców opiera się na schemacie Hornera?

▮ Algorytm Rabina-Karpa

Schemat Hornera pozwala obliczać **wartość wielomianu** (hashowanie toczące) efektywnie w $O(m)$:

$$h = (c_1 \cdot d^{m-1} + c_2 \cdot d^{m-2} + \dots + c_m) \bmod q$$

Kluczowa właściwość: przy przesunięciu okna o jedną pozycję nowy hash obliczamy w $O(1)$, odejmując wpływ pierwszego znaku i dodając nowy.

Pytanie 10.2

W jakim algorytmie może wystąpić „fałszywe trafienie” (spurious hit)?

▮ W algorytmie Rabina-Karpa

Fałszywe trafienie (spurious hit) zachodzi, gdy **hashe są równe, ale ciągi znaków różne** (kolizja funkcji haszującej). Po każdym dopasowaniu hashy konieczna jest weryfikacja przez bezpośrednie porównanie – $O(m)$. W praktyce fałszywych trafień jest mało, więc algorytm działa w $O(n + m)$ oczekiwanie.

11. Algorytmy zrandomizowane (teoria)

Pytanie 11.1

Jaki rodzaj czasu działania bada się w algorytmach zrandomizowanych?

▮ Oczekiwany czas działania (expected running time)

W odróżnieniu od algorytmów deterministycznych:

- Analiza worst-case dotyczy konkretnego wejścia
- Analiza oczekiwana dotyczy losowych wyborów algorytmu (nie rozkładu wejść)

Oczekiwany czas = $E[T(n)]$ uśredniony po wszystkich możliwych wynikach rzutów monetą / losowaniach dokonywanych przez algorytm – niezależnie od tego, jakie dane dostanie.

Pytanie 11.2

W jakim celu stosuje się randomizację w algorytmach?

Randomizacja służy kilku ważnym celom:

Cel	Przykład
Unikanie najgorszego przypadku	Losowy pivot w Quicksort eliminuje $O(n^2)$ dla posortowanych danych
Uproszczenie algorytmu	Hashing zamiast złożonych struktur deterministycznych
Przełamywanie symetrii	Algorytmy rozproszone (np. wybór lidera)
Przyspieszenie w praktyce	Randomizowane algorytmy często szybsze niż deterministyczne odpowiedniki
Odporność na złośliwe wejścia	Adversary nie zna losowych wyborów algorytmu

Kluczowa idea: nie liczymy na szczęście w danych wejściowych – sami wprowadzamy losowość do algorytmu.